САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Домашняя работа №5

Выполнил:

Кащеев Максим

Группа

J3211

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках домашней работы №5 необходимо установить и настроить окружение для разработки на Vue.js с использованием менеджера зависимостей npm, создать простое одностраничное приложение Notes app на Vue 3 с использованием Vite, Vue Router, Pinia, axios и Bootstrap.

Ход работы

Установка и настройка окружения
Была установлена глобальная поддержка Vue и создан базовый проект на Vite:

- установлены зависимости, указанные в package.json: vue, vue-router, pinia, axios, bootstrap и dev-зависимости для Vite;
- проверена структура проекта: в корне папки vue-project расположены файлы package.json, vite.config.js, index.html и папка src с исходным кодом приложения.

Настройка маршрутизации и состояния

- Настроен Vue Router в файле src/router/index.js: создан маршрутизатор с историей createWebHistory и одним маршрутом path '/' на компонент NotesPage.vue.
- Настроено хранилище Pinia в src/stores/index.js (создан экземпляр pinia) и в src/stores/notes.js (создан store useNotesStore). В сторе реализованы:
  state с массивом notes;
  action loadNotes() для загрузки списка заметок с сервера;
  action createNote(data) для отправки новой заметки на сервер.

Настройка API-слоя

- В папке src/api создан файл instance.js с настройкой axios: baseURL установлен на http://localhost:3000; создан и экспортирован экземпляр axios.

В файле src/api/notes.js реализован класс NotesApi с методами: getAll() - GET /notes для получения всех заметок; -reateNote(data) - POST /notes для создания новой заметки.

В src/api/index.js создан и экспортирован объект notesApi как экземпляр NotesApi, получающий на вход настроенный axios instance.

Создание компонентов и layout

Создан компонент NoteCard.vue (src/components/NoteCard.vue), который принимает props name и text и выводит их в карточке Bootstrap.

Создан базовый layout BaseLayout.vue (src/layouts/BaseLayout.vue), который оборачивает содержимое в контейнер Bootstrap и через слот отображает переданные дочерние элементы.

В корневом компоненте App.vue настроено отображение <router-view />, чтобы рендерить текущую страницу маршрута.

Реализация страницы NotesPage

В файле src/views/NotesPage.vue создан компонент NotesPage, который:
подключает BaseLayout и NoteCard;
через Pinia (mapState, mapActions и useNotesStore) получает доступ к notes и действиям loadNotes и createNote;
содержит форму с полями name и text, связанной с объектом form через v-model;
обрабатывает отправку формы методом createCard(), который вызывает createNote(form), затем loadNotes() и сбрасывает форму;
в шаблоне выводит список заметок с помощью v-for, передавая name и text в NoteCard.

В хуке mounted() вызывается loadNotes(), чтобы при загрузке страницы сразу отобразить текущие заметки с сервера.

Настройка стилей и Bootstrap

В main.js подключены стили Bootstrap и js-часть Bootstrap, а также пользовательский файл стилей main.css из папки src/assets.

В шаблоне NotesPage.vue к форме и элементам ввода добавлены классы Bootstrap (form-control, d-flex, my-5 и т.п.) для получения вида, совпадающего с образцом из методички.

Настройка и запуск json-server

В корне проекта был создан файл db.json с коллекцией "notes" (массив объектов с полями id, name, text, userId).

Установлен и запущен json-server командой:
npm install -D json-server
npx json-server --watch db.json --port 3000

После запуска json-server endpoint http://localhost:3000/notes стал доступен для GET и POST запросов из приложения.

апуск и тестирование приложения

Frontend-часть запускается командой npm run dev (или npm start).

В браузере открыта страница по адресу, выведенному Vite

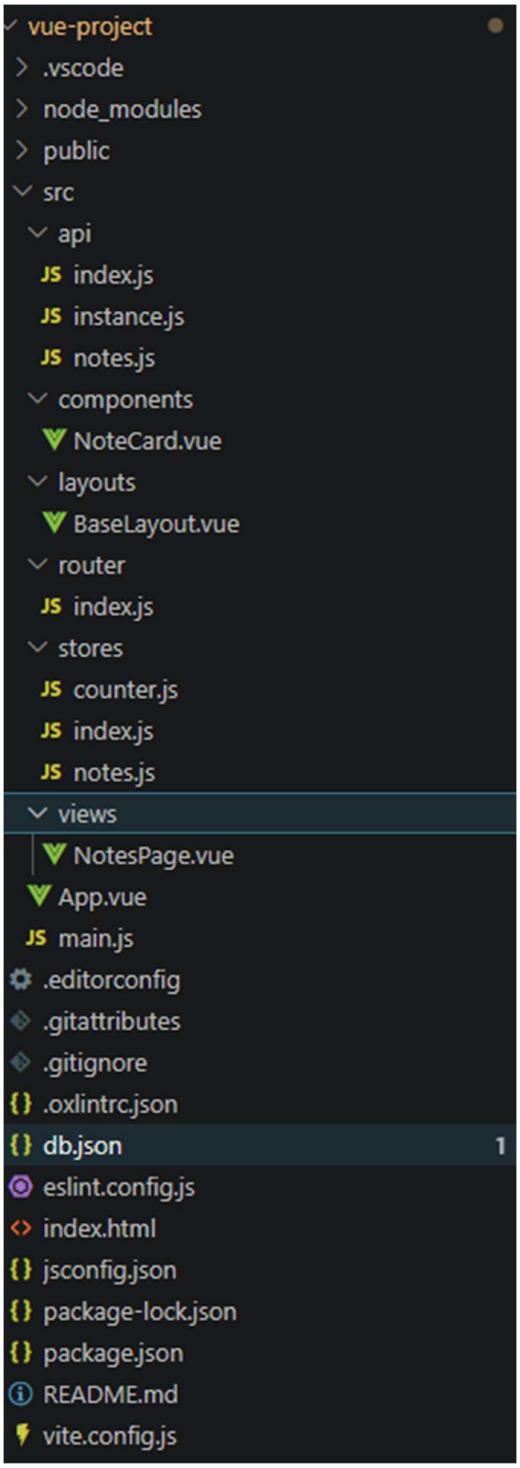
При загрузке страницы автоматически подгружались существующие заметки и отображались под формой.

При вводе заголовка и текста заметки и нажатии кнопки отправить отправлялся POST-запрос на json-server, затем приложение повторно загружало список заметок и отображало новую заметку внизу списка.

Вывод

В ходе выполнения работы было освоено создание Vue 3-приложения с использованием Vite и npm, настройка маршрутизации через Vue Router, управление состоянием через Pinia и организация API-слоя на базе axios. Был развернут тестовый backend (json-server) на http://localhost:3000 для хранения заметок и реализованы запросы GET и POST к ресурсу /notes. В результате приложение Notes app позволяет загружать список заметок с сервера и добавлять новые заметки через форму, после чего они отображаются на странице, что подтверждает корректную работу связки Vue + Router + Pinia + axios + json-server.

Приложения:



Скриншот 1: файловая структура

```
ject > {} db.json > ...
{
  "notes": [
    {
      "id": "1",
      "name": "Первая заметка",
      "text": "Тестовая запись",
      "userId": 1
    },
    {
      "name": "123",
      "text": "123",
      "userId": 1,
      "id": "ukBcFAiIH1M"
    }
  ],
  "$schema": "./node_modules/json-server/schema.json"
}
```

Скриншот 2: db.json

```
oject > src > api > JS index.js > ...
import instance from '@/api/instance'
import NotesApi from '@/api/notes'

const notesApi = new NotesApi(instance)

export {
  notesApi
}
```

Скриншот 3: src/api/index.js

```
vue-project > src > api > JS instance.js > ...
1  import axios from 'axios'
2
3  const apiURL = 'http://localhost:3000'
4
5  const instance = axios.create({
6    baseURL: apiURL
7  })
8
9  export default instance
```

Скриншот 4: src/api/instance.js

vue-project > src > api > JS notes.js > NotesApi

```js
class NotesApi {
  constructor(instance) {
    this.API = instance
  }

  getAll = async () => {
    return this.API({
      url: '/notes'
    })
  }

  createNote = async (data) => {
    return this.API({
      method: 'POST',
      url: '/notes',
      data,
      headers: {
        'Content-Type': 'application/json'
      }
    })
  }
}

export default NotesApi
```

Скриншот 5: src/api/notes.js

vue-project > src > components > NoteCard.vue

```
1   <template>
2     <div class="card">
3       <div class="card-body">
4         <h5 class="card-title">{{ name }}</h5>
5         <p class="card-text">
6           {{ text }}
7         </p>
8       </div>
9     </div>
10  </template>
11
12  <script>
13  export default {
14    name: 'NoteCard',
15    props: {
16      name: {
17        type: String,
18        required: true
19      },
20      text: {
21        type: String,
22        required: false,
23        default: ''
24      }
25    }
26  }
27  </script>
```

Скриншот 6: src/components/NoteCard.vue

vue-project > src > layouts > BaseLayout.vue

```
1   <template>
2     <main class="container my-2">
3       <slot />
4     </main>
5   </template>
6
7   <script>
8   export default {
9     name: 'BaseLayout'
10  }
11  </script>
```

Скриншот 7: src/layouts/BaseLayout.vue

```
ue-project > src > router > JS index.js > default
1   import { createRouter, createWebHistory } from 'vue-router'
2
3   const router = createRouter({
4     history: createWebHistory(import.meta.env.BASE_URL),
5     routes: [
6       {
7         path: '/',
8         name: 'notes',
9         component: () => import('@/views/NotesPage.vue')
10      }
11    ]
12  })
13
14  export default router
```

Скриншот 8: src/router/index.js

```
import { ref, computed } from 'vue'
import { defineStore } from 'pinia'

export const useCounterStore = defineStore('counter', () =>
  const count = ref(0)
  const doubleCount = computed(() => count.value * 2)
  function increment() {
    count.value++
  }

  return { count, doubleCount, increment }
})
```

Скриншот 9: src/stores/counter.js

```
vue-project > src > stores > JS index.js > ...
1   import { createPinia } from 'pinia'
2
3   const pinia = createPinia()
4
5   export default pinia
```

Скриншот 10: src/stores/index.js

```
vue-project > src > stores > JS notes.js > ...
1   import { defineStore } from 'pinia'
2   import { notesApi } from '@/api'
3
4   const useNotesStore = defineStore('notes', {
5     state: () => ({
6       notes: []
7     }),
8
9     actions: {
10      async loadNotes() {
11        const response = await notesApi.getAll()
12        this.notes = response.data
13        return response
14      },
15
16      async createNote(data) {
17        const response = await notesApi.createNote(data)
18        return response
19      }
20    }
21  })
22
23  export default useNotesStore
```

Скриншот 11: src/stores/notes.js

```
<template>
  <BaseLayout>
    <h1>Notes app</h1>

    <form
      ref="noteForm"
      @submit.prevent="createCard"
      class="d-flex flex-column my-5"
    >
      <input
        type="text"
        v-model="form.name"
        class="form-control my-1"
        placeholder="Название заметки"
      >

      <textarea
        cols="30"
        rows="10"
        v-model="form.text"
        class="form-control my-1"
        placeholder="Текст заметки"
      ></textarea>

      <button
        type="submit"
        class="btn btn-primary mt-2"
      >
        Отправить
      </button>
    </form>

    <div class="row row-cols-1 row-cols-md-2 g-4 mt-5" id="notes">
      <div class="col" v-for="note in notes" :key="note.id">
        <NoteCard :name="note.name" :text="note.text" />
      </div>
    </div>
  </BaseLayout>
</template>

<script>
import { mapActions, mapState } from 'pinia'
import useNotesStore from '@/stores/notes'
import BaseLayout from '@/layouts/BaseLayout.vue'
import NoteCard from '@/components/NoteCard.vue'

export default {
  name: 'NotesPage',
```

Скриншот 12: src/views/NotesPage.vue

```
49
50    components: {
51      BaseLayout,
52      NoteCard
53    },
54
55    data() {
56      return {
57        form: {
58          name: '',
59          text: '',
60          userId: 1
61        }
62      }
63    },
64
65    computed: {
66      ...mapState(useNotesStore, ['notes'])
67    },
68
69    mounted() {
70      this.loadNotes()
71    },
72
73    methods: {
74      ...mapActions(useNotesStore, ['loadNotes', 'createNote']),
75
76      async createCard() {
77        await this.createNote(this.form)
78        await this.loadNotes()
79
80        this.form = {
81          name: '',
82          text: '',
83          userId: 1
84        }
85      }
86    }
87  }
88  </script>
```

Скриншот 13: src/views/NotesPage.vue

```
vue-project > src > App.vue
1  <template>
2    <router-view />
3  </template>
4
5  <script>
6  export default {
7    name: 'App'
8  }
9  </script>
```

Скриншот 14: src/App.vue

```
vue-project > src > main.js > ...
1   import { createApp } from 'vue'
2
3   import App from '@/App.vue'
4   import router from '@/router'
5   import store from '@/stores'
6
7   import 'bootstrap/dist/css/bootstrap.min.css'
8   import 'bootstrap'
9
10  const app = createApp(App)
11
12  app.use(store)
13  app.use(router)
14
15  app.mount('#app')
```

Скриншот 15: src/main.js

vue-project > vite.config.js > default

```
1   import { fileURLToPath, URL } from 'node:url'
2   import { defineConfig } from 'vite'
3   import vue from '@vitejs/plugin-vue'
4
5   export default defineConfig({
6     plugins: [vue()],
7     resolve: {
8       alias: {
9         '@': fileURLToPath(new URL('./src', import.meta.url))
10      }
11    }
12  })
```

Скриншот 16: vite.config.js